

## ส่วนที่ 1: หลักการออกแบบ

- อธิบายว่าใช้ SOLID ส่วนไหนบ้างและเกี่ยวข้องกับ Entity อย่างไร

ตอบ 1.1 ใช้ SRP โดยใช้แบ่งหน้าที่แต่ละคลาสอย่างชัดเจน โดยจะมีการแยก Product, ProductDetail, Review, Service, Controller และ Repository ออกจากกันรับผิดชอบแต่ละหน้าที่ โดย Product ทำหน้าที่เก็บข้อมูลหลักของสินค้า เช่น ชื่อสินค้า ราคา ยี่ห้อ และจำนวนสินค้า ProductDetail ทำหน้าที่เก็บรายละเอียดเพิ่มเติมของสินค้า เช่น รายละเอียดสินค้า การรับประกัน น้ำหนัก และขนาด Review ทำหน้าที่เก็บข้อมูลรีวิว เช่น ผู้รีวิว คะแนน และความคิดเห็น

1.2 ใช้ OCP การเพิ่มฟังก์ชันใหม่โดยไม่ยุ่งเกี่ยวกับโค้ดเดิม โดยมี DiscountStrategy เป็นตัวกำหนดว่าการคำนวณส่วนลดต้องมีอะไรบ้าง แล้วสร้าง Strategy แยกออกมา NoDiscountStrategy, MemberDiscountStrategy, SeasonalSaleStrategy ซึ่งถ้าในอนาคตต้องการเพิ่มส่วนลดแบบอื่นก็สามารถสร้างคลาสเพิ่มมาได้โดยไม่ต้องแก้ไขโค้ดอันเก่าที่เขียนไว้แล้วให้ยุ่งยาก

1.5 ใช้ LSP โดยมี DiscountStrategy เป็น Interface และมี MemberDiscountStrategy, SeasonalSaleStrategy, NoDiscountStrategy สามารถนำมาใช้เป็น DiscountStrategy ได้ โดย DiscountContext ไม่จำเป็นต้องรู้รายละเอียดภายในของแต่ละ Strategy

1.4 ใช้ ISP โดยไม่ควรบังคับให้คลาสต้อง implement method ที่ตัวเองไม่ได้ใช้งาน ซึ่งจะมี การแยก Repository ตาม Entity เช่น ProductRepository, ProductDetailRepository, ReviewRepository ซึ่ง แต่ละ Repository จัดการข้อมูลของ Entity ที่ตัวเองรับผิดชอบ ทำให้ไม่ต้องสร้าง Repository ที่รวมทุกอย่างไว้

1.5 ใช้ DIP โดย ใน ProductService เราไม่ได้สร้าง ProductRepository ขึ้นมาเอง แต่รับเข้ามาทาง Constructor ทำให้ ProductService ทำงานผ่าน ProductRepository และไม่ต้องรู้รายละเอียดว่าการติดต่อ database ทำงานอย่างไร

- อธิบายความแตกต่างของ 1:1 กับ 1:N — ใช้กรณีไหนควรใช้แบบไหน

ตอบ ความสัมพันธ์ 1:1 หมายถึง ข้อมูลหนึ่งรายการสามารถมีข้อมูลอีกประเภทได้เพียงหนึ่งรายการ โดยจะใช้ในกรณี ที่ข้อมูลสองอันมีความสัมพันธ์กันแบบหนึ่งต่อหนึ่งจริง ๆ เช่น สินค้าหนึ่งชิ้นมีรายละเอียดเฉพาะของตัวเองหนึ่งชุด

ความสัมพันธ์ 1:N หมายถึง ข้อมูลหนึ่งรายการสามารถมีข้อมูลอีกประเภทได้หลายรายการ โดยจะใช้ในกรณี ที่ข้อมูลหนึ่งรายการสามารถมีข้อมูลที่เกี่ยวข้องได้หลายรายการ เช่น สินค้าหนึ่งชิ้นมีรีวิวหลายคน

- อธิบาย Strategy Pattern สำหรับคำนวณส่วนลด

ตอบ มี DiscountStrategy เป็น Interface โดย NoDiscountStrategy จะ implement มา ซึ่งไม่มีส่วนลดจะ return ราคาปกติ ส่วน MemberDiscountStrategy จะ implement มา และคำนวณส่วนลดซึ่งจะลด 10 % โดยจะ return  $price * 0.90$  และ SeasonalSaleStrategy ก็ จะ implement มาเหมือนกันโดยจะคำนวณส่วนลด ซึ่งจะลด 20 % โดยจะ return  $price * 0.80$  จากนั้น DiscountContext จะเป็นตัวเรียก Strategy ที่ถูกเลือก โดยถ้าภายหลังต้องการเพิ่ม ส่วนลดแบบใหม่ เราสามารถสร้าง Strategy ใหม่เพิ่มได้ โดยไม่ต้องเอาโค้ดการคำนวณทุกแบบมาเขียนรวมกัน

- อธิบาย Execution Flow ตั้งแต่ HTTP Request → DB

ตอบ 1. ผู้ใช้ส่ง HTTP Request เช่น ผู้ใช้เดิหน้าเว็บ /products หรือกดปุ่ม Save ในหน้าเพิ่มสินค้า POST /products/save

2. Controller รับ Request โดย ProductController เป็นตัวรับ Request เช่น เมื่อผู้ใช้กด Save ข้อมูลจาก HTML จะถูกนำมาใส่ใน Object Product แล้ว Controller ส่ง Product ไปให้ ProductService

3. Service จัดการ Business Logic โดย ProductService จะเป็นส่วนที่จัดการการทำงานของระบบ เช่น ตอนบันทึกสินค้า จะตรวจสอบ ProductDetail และ Review และ เพื่อให้ความสัมพันธ์ระหว่าง Entity ถูกต้อง นอกจากนี้ Service ยังใช้ Strategy สำหรับคำนวณราคาส่วนลด

4. Repository ทำการติดต่อฐานข้อมูล โดย Service จะเรียก ProductRepository เป็นส่วนที่ติดต่อกับ ฐานข้อมูลผ่าน Spring Data JPA เราไม่จำเป็นต้องเขียน SQL สำหรับการบันทึกข้อมูลเอง

5. JPA/Hibernate จัดการข้อมูล โดย Spring Data JPA และ Hibernate จะนำ Object ที่เราส่งเข้าไปแปลง เป็นคำสั่งที่ใช้กับฐานข้อมูล จากนั้นข้อมูลจะถูกบันทึกลง PostgreSQL

6. ข้อมูลถูกเก็บใน PostgreSQL โดยข้อมูลจะอยู่ในตาราง 3 ตาราง products product\_details, reviews และมีการกำหนดความสัมพันธ์ตามที่โจทย์กำหนด

## ส่วนที่ 2: Code + คำอธิบาย

### 1. Product Entity

```
Java
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String category;
    private String brand;
    private Integer stock;
    private Double price;
    private String discountType;

    @Transient
    private Double discountedPrice;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(
        name = "detail_id",
        referencedColumnName = "id"
    )
    private ProductDetail detail;

    @OneToMany(
        mappedBy = "product",
        cascade = CascadeType.ALL
    )
    private List<Review> reviews = new ArrayList<>();
}
```

#### อธิบาย Annotation

- @Entity ทำหน้าที่บอก JPA คลาสนี้เป็น Entity ที่ต้องจัดการกับ Database ดังนั้น Product จะถูกนำไปสร้างเป็นตาราง products
- @Table(name = "products") จะกำหนดชื่อ Table ใน Database เป็น products
- @Id กำหนด id ให้เป็น Primary Key ของ Product
- @GeneratedValue ให้ Database สร้างค่า ID อัตโนมัติ
- @Transient หมายความว่า discountedPrice ไม่ถูกเก็บเป็น Column ใน Database มันใช้สำหรับเก็บค่าที่คำนวณขึ้นมาเพื่อแสดงผล
- @OneToOne กำหนดความสัมพันธ์ 1:1
- @JoinColumn กำหนด FK ของ Product โดย name = "detail\_id" คือชื่อ FK Column ใน products และ referencedColumnName = "id" คือ Column ที่ถูกอ้างอิงใน product\_details
- @OneToMany กำหนดว่า Product หนึ่งตัวมี Review ได้หลายตัว

- mappedBy = "product" หมายถึงความสัมพันธ์นี้ถูกกำหนดโดย field หมายถึงความสัมพันธ์นี้ถูกกำหนดโดย field ที่อยู่ใน Review ดังนั้น Review จะมี FK

## 2. Product Entity

```
Java
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String description;
    private String warranty;
    private Double weight;
    private String dimensions;
    private String manufacturedCountry;

    @OneToOne(mappedBy = "detail")
    private Product product;
}
```

### อธิบาย Annotation

- @Entity ทำหน้าที่บอก JPA คลาสนี้เป็น Entity ที่ต้องจัดการกับ Database ดังนั้น ProductDetail จะถูกนำไปสร้างเป็นตาราง product\_details
- @Table กำหนด Table ใน Database เป็น product\_details
- @Id กำหนด id ให้เป็น Primary Key ของ ProductDetail
- @GeneratedValue ให้ Database สร้างค่า ID อัตโนมัติ
- @OneToOne แสดงว่า ProductDetail มีความสัมพันธ์แบบ 1:1 กับ Product
- mappedBy = "detail" หมายความว่า Product เป็นเจ้าของ Relationship เพราะใน Product มี เพราะใน Product และ @JoinColumn(name = "detail\_id") ดังนั้น ProductDetail ไม่ต้องสร้าง @JoinColumn ขึ้น

### 3. Review Entity

```
Java
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String reviewer;
    private Integer rating;
    private String comment;
    private LocalDate reviewDate;

    @ManyToOne
    @JoinColumn(name = "product_id")
    private Product product;
}
```

#### อธิบาย Annotation

- @Entity ทำหน้าที่บอก JPA คลาสนี้เป็น Entity ที่ต้องจัดการกับ Database ดังนั้น Review จะถูกนำไปสร้างเป็นตาราง reviews
- @Table กำหนด Table ใน Database เป็น reviews
- @Id กำหนด id ให้เป็น Primary Key
- @GeneratedValue ให้ Database สร้างค่า ID อัตโนมัติ
- @ManyToOne หมายถึง Review หลายรายการสามารถเป็นของ Product เดียวกันได้
- @JoinColumn สร้าง FK reviews.product\_id products.id

### 4. ProductService

```
Java
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
}
```

## อธิบาย Annotation

- @Service บอก Spring ว่าคลาสนี้เป็น Service Layer

```
Java
public List<Product> getAllProducts() {

    List<Product> products =
        productRepository.findAll();

    for (Product product : products) {
        calculateDiscountedPrice(product);
    }

    return products;
}
```

- getAllProducts() คือการหา Product ทั้งหมด คำนวณราคาหลังส่วนลดและส่งกลับ Controller

```
Java
public Product saveProduct(Product product) {

    if (product.getDetail() != null) {
        product.getDetail().setProduct(product);
    }

    if (product.getReviews() != null) {

        for (Review review : product.getReviews()) {

            review.setProduct(product);

            if (review.getReviewDate() == null) {
                review.setReviewDate(LocalDate.now());
            }
        }
    }

    return productRepository.save(product);
}
```

- หน้าที่หลักคือเตรียม Relationship ก่อนบันทึก แล้ว productRepository.save(product); บันทึกลง Database

```

Java
if ("MEMBER".equalsIgnoreCase(
    product.getDiscountType())) {

    context = new DiscountContext(
        new MemberDiscountStrategy()
    );

}
else if ("SEASONAL".equalsIgnoreCase(
    product.getDiscountType())) {

    context = new DiscountContext(
        new SeasonalSaleStrategy()
    );

}
else {

    context = new DiscountContext(
        new NoDiscountStrategy()
    );

}

```

- เป็นเงื่อนไขการเลือก Strategy ว่าเป็นประเภทไหนควรได้ราคาเท่าไรตามประเภท

## 5. ProductController

```

Java
@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(
        ProductService productService) {

        this.productService = productService;
    }

}

```

- @Controller บอก Spring ว่าคลาสนี้เป็น Controller มีหน้าที่รับ HTTP Request จาก Browser
- @RequestMapping("/products") จะกำหนด URL หลัก /products ดังนั้น @GetMapping จะเป็น GET /products แสดงสินค้า , @GetMapping("/add") จะเป็น GET /products/add เพิ่มสินค้า , @PostMapping("/save") จะเป็น /products/save บันทึกสินค้า , @GetMapping("/edit/{id}") เป็น /products/edit/{id} เปิดหน้าแก้ไขตาม id , @PostMapping("/update/{id}") เป็น /products/update/{id} อัปเดต , @GetMapping("/delete/{id}") เป็น /products/delete/{id} เปิดหน้ายืนยันลบ , @PostMapping("/delete/{id}") เป็น /products/delete/{id} ลบข้อมูล

## 6. Constructor Injection

### 1. ใน ProductService

```
Java
private final ProductRepository productRepository;

public ProductService(
    ProductRepository productRepository) {

    this.productRepository = productRepository;
}
```

โดย Spring จะส่ง ProductRepository เข้ามาให้ ProductService

### 2. ใน ProductController

```
Java
private final ProductService productService;

public ProductController(
    ProductService productService) {

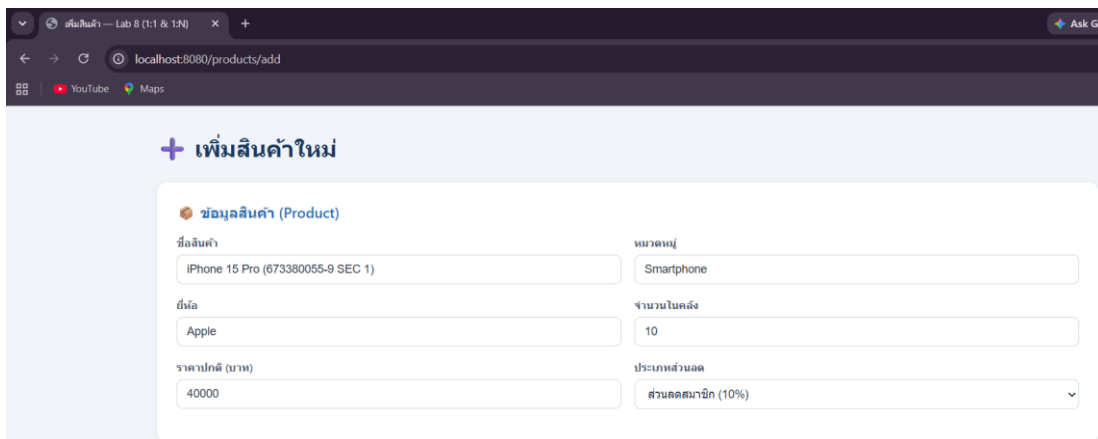
    this.productService = productService;
}
```

Spring ส่ง ProductService เข้ามา

โดย Service ไม่ต้องสร้าง Object ของ Repository หรือ DiscountContext ขึ้นมาเอง แต่ให้ Spring เป็นคนจัดการ และส่ง Object ที่ต้องการเข้ามา ทำให้โค้ดแยกหน้าที่กันชัดเจนและทดสอบได้ง่ายขึ้น

## ส่วนที่ 3: ภาพหน้าจอ

- กรอกข้อมูล Product + ProductDetail



The screenshot shows a web browser window with the URL `localhost:8080/products/add`. The page has a header with a purple plus icon and the text '+ เพิ่มสินค้าใหม่'. Below the header is a form titled 'ข้อมูลสินค้า (Product)'. The form contains several input fields and a dropdown menu:

- ชื่อสินค้า**: iPhone 15 Pro (673380055-9 SEC 1)
- หมวดหมู่**: Smartphone
- ยี่ห้อ**: Apple
- จำนวนในคลัง**: 10
- ราคาปกติ (บาท)**: 40000
- ประเภทส่วนลด**: ส่วนลดสมาชิก (10%)



ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

Smartphone with A17 Pro chip

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

0.18

ขนาด (กว้าง x ยาว x สูง)

146.6 x 70.6 x 8.25 mm

ประเทศที่ผลิต

China

- รายการสินค้าในตาราง

**Product Shop** Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

| # | ชื่อสินค้า                        | หมวดหมู่   | ยี่ห้อ | คลัง | ราคา (บาท) | ส่วนลด | ราคาสุทธิ | วันประกัน (1:1) | น้ำหนัก (1:1) | รีวิว (1:N) | จัดการ      |
|---|-----------------------------------|------------|--------|------|------------|--------|-----------|-----------------|---------------|-------------|-------------|
| 1 | iPhone 15 Pro (673380055-9 SEC 1) | Smartphone | Apple  | 10   | 40000.00   | MEMBER | 36000.00  | 1 Year          | 0.18          | 1 รีวิว     | แก้ไข<br>ลบ |

- หน้าฟอร์มแก้ไข

**แก้ไขสินค้า**

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 15 Pro (673380055-9 SEC 1)

หมวดหมู่

Smartphone

ยี่ห้อ

Apple

จำนวนในคลัง

10

ราคาปกติ (บาท)

50000.0

ประเภทส่วนลด

ส่วนลดสมาชิก (10%)

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

Smartphone with A17 Pro chip

ระยะเวลาประกัน

2 Year

น้ำหนัก (กิโลกรัม)

0.18

ขนาด

146.6 x 70.6 x 8.25 mm

ประเทศที่ผลิต

China

บันทึกการแก้ไข ยกเลิก

**Product Shop** Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

| # | ชื่อสินค้า                        | หมวดหมู่   | ยี่ห้อ | คลัง | ราคา (บาท) | ส่วนลด | ราคาสุทธิ | รับประกัน (1:1) | น้ำหนัก (1:1) | รีวิว (1:N) | จัดการ  |
|---|-----------------------------------|------------|--------|------|------------|--------|-----------|-----------------|---------------|-------------|---------|
| 1 | iPhone 15 Pro (673380055-9 SEC 1) | Smartphone | Apple  | 10   | 50000.00   | MEMBER | 45000.00  | 2 Year          | 0.18          | 1 รีวิว     | แก้ไขลบ |

- ยืนยันลบ + ห้างลบ

**ยืนยันการลบสินค้า**

คุณต้องการลบสินค้านี้ใช่ไหม? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า: iPhone 15 Pro (673380055-9 SEC 1)  
 หมวดหมู่: Smartphone  
 ยี่ห้อ: Apple  
 ราคา: 50000.00 บาท  
 รายละเอียด: Smartphone with A17 Pro chip  
 รับประกัน: 2 Year

⚠ ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกันด้วย เนื่องจาก CascadeType.ALL

ยืนยันลบ ยกเลิก

**Product Shop** Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

| # | ชื่อสินค้า | หมวดหมู่ | ยี่ห้อ | คลัง | ราคา (บาท) | ส่วนลด | ราคาสุทธิ | รับประกัน (1:1) | น้ำหนัก (1:1) | รีวิว (1:N) | จัดการ |
|---|------------|----------|--------|------|------------|--------|-----------|-----------------|---------------|-------------|--------|
|---|------------|----------|--------|------|------------|--------|-----------|-----------------|---------------|-------------|--------|

ยังไม่มีสินค้า — กด "+" เพิ่มสินค้าใหม่" เพื่อเริ่มต้น

- pgAdmin แสดง 3 ตาราง (products, product\_details, reviews) พร้อม FK

pgAdmin 4

Object Explorer

- Materialized views
- Operators
- Procedures
- Sequences
- Tables (3)
  - product\_details
    - Columns
    - Constraints (1)
      - product\_details\_pkey
    - Indexes
    - RLS Policies
    - Rules
    - Triggers
  - products
    - Columns

Query

```
SELECT * FROM public.product_details
ORDER BY id ASC
```

Data Output

| id | description                  | dimensions             | manufactured_country | warranty | weight |
|----|------------------------------|------------------------|----------------------|----------|--------|
| 1  | Smartphone with A17 Pro c... | 146.6 x 70.6 x 8.25 mm | China                | 1 Year   | 0.18   |
| 2  | Smartphone with A17 Pro c... | 146.6 x 70.6 x 8.25 mm | China                | 2 Year   | 0.18   |

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Tables (3)
  - product\_details
    - Columns
    - Constraints (1)
      - product\_details\_pkey
    - Indexes
    - RLS Policies
    - Rules
    - Triggers
  - products
    - Columns
    - Constraints (3)
      - fkca9f512fdist3ot4ix883w8bcj
      - products\_pkey
      - ukfq6xkbp2dry2wsa6wcuhsjcwj

public.products/lab8shop/postgres@PostgreSQL 18

Query

```
1 SELECT * FROM public.products
2 ORDER BY id ASC
```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

| id          | brand                   | category                | discount_type           | name                    | price            | stock   | detail_id |
|-------------|-------------------------|-------------------------|-------------------------|-------------------------|------------------|---------|-----------|
| [PK] bigint | character varying (255) | character varying (255) | character varying (255) | character varying (255) | double precision | integer | bigint    |
| 1           | Apple                   | Smartphone              | SEASONAL                | Ipad pro 2021           | 45000            | 20      | 4         |

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Constraints
- Domains
- FTS Configurations
- FTS Dictionaries
- FTS Parsers
- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (3)
  - product\_details
    - Columns
    - Constraints (2)
      - fkpl51cejpw4gy5swfar8br9ngj
      - reviews\_pkey
  - products
  - reviews
    - Columns
    - Constraints (2)
      - fkpl51cejpw4gy5swfar8br9ngj
      - reviews\_pkey

public.reviews/lab8shop/postgres@PostgreSQL 18

Query

```
1 SELECT * FROM public.reviews
2 ORDER BY id ASC
```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

| id          | comment                 | rating  | review_date | reviewer                | product_id |
|-------------|-------------------------|---------|-------------|-------------------------|------------|
| [PK] bigint | character varying (255) | integer | date        | character varying (255) | bigint     |
| 1           | cse                     | 5       | 2026-09-09  | awons                   | 2          |